

Scientific Computing using Python 1

Python and Scientific Computing (2025)

Marcell Richard Fekete
Aalborg University
Department of Computer Science
mrfe@cs.aau.dk

July 2025

1 Introduction

Atmospheric convection represents the movement of heat and moisture in the atmosphere in a vertical manner, often accompanied by various cloud formations. The Lorenz system or Lorenz attractor, devised by Edward Lorenz, is a mathematical model made up of three ordinary differential equations (ODE), originally proposed for this chaotic system of atmospheric convection. The Lorenz attractor has developed additional uses as well, including simulating weather and price fluctuations in a stock exchange. The systems the Lorenz attractor simulates involve variables such as temperature and physical motion which change continuously. Since computers cannot handle continuous change, some effort has to be made to discretise the ODE in the Lorenz attractor.

In this project, I apply Euler's method to initialise the positions of point in dimension $\{x, y, z\}$ and simulate continuous change of the positions via pre-defined steps, then representing the results in 2 and 3 dimensions I run the simulation for five sets of parameter sets, *i.e.*, cases – described in Section 2.3 –, and save the positions and plot them for each case¹.

2 Mathematical Modelling

2.1 Lorenz Attractor

As mentioned in Section 1, the Lorenz attractor contains three ODE:

¹The repository can be found at https://github.com/vrmer/scientific_computing_1.git.

$$\begin{aligned}
\frac{dx}{dt} &= \sigma(y - x) \\
\frac{dy}{dt} &= x(\rho - z) - y \\
\frac{dz}{dt} &= xy - \beta z
\end{aligned} \tag{1}$$

where $\{x, y, z\}$ are positions of a particle in these dimensions, t expresses time, and $\{\sigma, \rho, \beta\}$ are parameters of the model. The ODE in Equation 1 are coupled, which means that the position in a given dimension is impacted by the positions in two or more dimensions. Dimensions $\{x, y\}$ matter for calculating x , and $\{x, y, z\}$ matter for calculating y and z .

2.2 Euler Solver

The simulation of the Lorenz attractor requires an ODE solver as well as the discretisation of time. Uniform sampling allows us to treat time as a sequence of discrete time steps:

$$t \leftarrow nt_\delta, \quad n \in \{0, 1, \dots, N\} \tag{2}$$

Furthermore, we can solve the ODE using the relatively simple Euler method. This approach works by approximating a general variable v with the following equation:

$$\frac{dv}{dt}((n+1)t_\delta) \cong \frac{v((n+1)t_\delta) - v(nt_\delta)}{t_\delta} \tag{3}$$

Using the formula in 3, we can denote v as $v(nt_\delta) = v[n]$ to rewrite the equations in 1:

$$\begin{aligned}
x[n+1] &\cong t_\delta \cdot \sigma(y[n] - x[n]) + x[n] \\
y[n+1] &\cong t_\delta(x(\rho - z) - y) \\
z[n+1] &\cong z + t_\delta(xy - \beta z)
\end{aligned} \tag{4}$$

The equations in 4 allow the calculation of the positions in $\{x, y, z\}$ in time step $n+1$ from the positions in time step n . Also see Algorithm 1.

Algorithm 1 Euler Method for ODE

Initialise $t = t_0, x = x_0, y = y_0, z = z_0$

while $t < t_{\text{end}}$ **do**

$x \leftarrow x + \Delta t \cdot \sigma(y - x)$

$y \leftarrow y + \Delta t \cdot (x \cdot (\rho - z) - y)$

$z \leftarrow z + \Delta t \cdot (x \cdot y - \beta \cdot z)$

$t \leftarrow t + t_\delta$

end while

return x, y, z

Case no.	σ	β	ρ
Case 1	10	$\frac{8}{3}$	6
Case 2	10	$\frac{8}{3}$	16
Case 3	10	$\frac{8}{3}$	28
Case 4	14	$\frac{8}{3}$	28
Case 5	14	$\frac{13}{3}$	28

Table 1: Lorenz attractor model parameter cases.

2.3 Parameter Cases

As per the assignment, I carry out the simulations for five model parameter groups in Table 1. Besides these parameters, the values for the initial position in dimensions $\{x, y, z\}$ has been fixed at $\{0.0, 1.0, 1.05\}$, the time step $t_\delta = 0.01$ and the maximum number of time steps $N = 50,000$.

3 Programming Solution

3.1 Euler Solver Implementation

The Euler solver implementation is a function following Algorithm 1 in and it is carried out in Python using the NumPy library.² Using NumPy arrays enables vectorising the processing of complex equations which makes computations more speedy and efficient. The individual equations for calculating $\{x, y, z\}$ in the current time step are all defined in separate functions in the script `src/euler_solver/equations.py` and imported to the function `solver` in `src/euler_solver/solver.py`.

```

1 def solver(
2     params,
3     output_path:
4     # Define output arrays
5     x_array = np.zeros(params.N)
6     y_array = np.zeros(params.N)
7     z_array = np.zeros(params.N)
8
9     # define the initial positions
10    x, y, z = params.x, params.y, params.z
11
12    # Simulate over N time steps
13    for i in range(params.N):
14        x = equation_x(x, y, params.sigma, params.t_delta)

```

²<https://numpy.org>

```

15         y = equation_y(x, y, z, params.rho, params.t_delta)
16         z = equation_z(x, y, z, params.beta, params.t_delta)
17
18         x_array[i] = x
19         y_array[i] = y
20         z_array[i] = z
21
22     if output_path:
23         save_arrays([x_array, y_array, z_array], output_path)
24
25     return x_array, y_array, z_array

```

The generated arrays are optionally saved in the .npz format, allowing the further processing of the values of the individual dimensions.

3.2 Providing Parameters

Parameters to the Euler solver are provided via a `Parameters` dataclass. This enables a generalised and flexible input to the `solver` function in Section 3.1. These include $\{x_0, y_0, z_0\}$ as parameters `x, y, z`, the model parameters $\{\sigma, \beta, \rho\}$ as parameters `sigma, beta, rho, t $\delta$` , *i.e.*, the size of the time step, and N , the total number of time steps to simulate.

These parameters are inputted using config files in the `toml` format³, thus ensuring ease of use and flexibility. These config files also provide the paths for saving the arrays and figures generated:

```

x = 0
y = 1
z = 1.05
sigma = 10
beta = 2.667
rho = 6

position_dir = "outputs/case_1"
figures_dir = "figures/case_1"

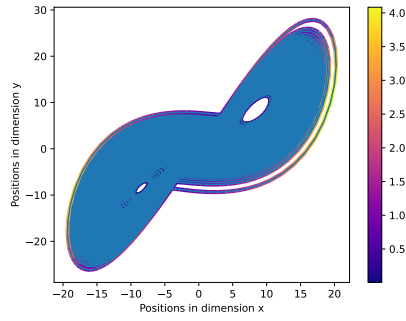
```

3.3 Visualisation

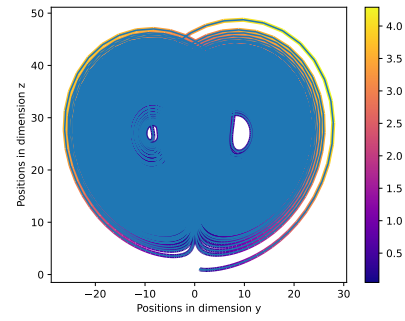
Simulations of the Lorenz attractor are visualised both in two and three dimensions using the `matplotlib` library⁴ (see Figure 1 for examples). Both functions take as input the file path where the arrays of the values for dimensions $\{x, y, z\}$ are saved and an optional output path

³<https://toml.io/en/>

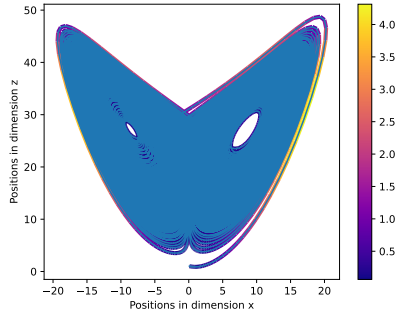
⁴<https://matplotlib.org>



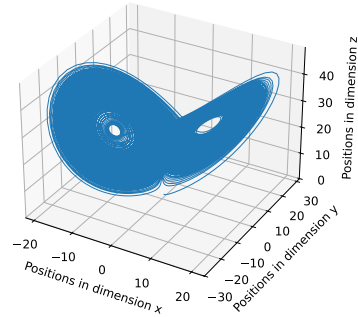
(a) Dimensions x, y



(b) Dimensions y, z



(c) Dimensions x, z



(d) Dimensions x, y, z

Figure 1: Visualisations representing various dimensions for Case 3 of the Lorenz attractor assignment ($x = 0, y = 1, z = 1.05, \sigma = 10, \beta = \frac{8}{3}, \rho = 28$). The 2D representations also include colouring representing the Euclidean distance between time steps.

where the figures are saved. Additionally, only the function for two-dimensional visualisation takes a tuple of size 2, allowing the representation of two of the three dimensions. In 2D figures, the Euclidean distance between time steps is visualised using a function adopted from the `matplotlib` library. The figures are saved as PDF, a high-quality format with relatively small file sizes.

3.4 Automated Testing

Tests evaluating the equations, the output of the solver, and the success of calculating the Euclidean distances for the visualisation are provided in the `src/tests` directory and can be run automatically using the `pytest` framework.⁵

4 Programming Solution by Generative AI

Besides implementing the Euler solver for the Lorenz attractor as described in Section 3, I also set out to compare the solution of generative AI as provided by a chatbot. This solution was generated by ChatGPT⁶ responding to the following prompt:

Prompt

Given parameters x , y , z and σ , β , and ρ , suggest a full working solution in Python for using Euler's method for solving the ODE of a Lorenz attractor.

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 # Lorenz system parameters
5 sigma = 10.0
6 beta = 8.0 / 3.0
7 rho = 6.0
8
9 # Time settings
10 dt = 0.01          # Time step
11 num_steps = 10000 # Number of steps
12
13 # Initialize arrays
14 x = np.zeros(num_steps)
15 y = np.zeros(num_steps)
16 z = np.zeros(num_steps)
17
18 # Initial conditions
19 x[0], y[0], z[0] = 0.0, 1.0, 1.05
```

⁵<https://docs.pytest.org/en/stable/>

⁶<https://chatgpt.com>

```

20
21 # Euler's method to solve the Lorenz system
22 for i in range(num_steps - 1):
23     dx = sigma * (y[i] - x[i])
24     dy = x[i] * (rho - z[i]) - y[i]
25     dz = x[i] * y[i] - beta * z[i]
26
27     x[i+1] = x[i] + dt * dx
28     y[i+1] = y[i] + dt * dy
29     z[i+1] = z[i] + dt * dz
30
31 # Plot the Lorenz attractor
32 fig = plt.figure(figsize=(10, 7))
33 ax = fig.add_subplot(111, projection='3d')
34 ax.plot(x, y, z, lw=0.5)
35 ax.set_title("Lorenz Attractor using Euler's Method")
36 ax.set_xlabel("X")
37 ax.set_ylabel("Y")
38 ax.set_zlabel("Z")
39 plt.show()

```

Given the same inputs – here tested with those of Case 1 –, the solution of ChatGPT arrives to the same 3D visualisation as my own. In their core logic, the two approaches are very similar. Both my and ChatGPT’s solution initialised the final arrays before adding the calculations, thus speeding up the process (see lines 14-16), and use vector operations for each time step (lines 22-29). In the latter case, the main difference is that I import separate functions for each equations, arguably improving modularity.

In general, the main difference in lies the modularity of the solutions. While this is not completely fair as a comparison since I have not specifically prompted ChatGPT to generate modular code, my repository provides testing, easy maintenance, and an ease of use via enabling interaction using config files.

5 Conclusion

In this project, I successfully implemented an Euler solver simulation for the Lorenz attractor and ran it for five different sets of parameters. My repository enables providing and retaining parameters via config files, it stores the simulation results, and creates plots in two and three dimensions, all stored in their own folders. The programming solution features automated testing. Finally, I directly compare the my own solver with that of ChatGPT.

A Use of Generative AI

I used ChatGPT to assist my own work in a few different ways. First, to supplant the use of official documentation in a subset of the cases, asking questions like how to save multiple arrays as an npz file, how to create a 3D plot using `matplotlib`, and how to create a Python module using `setup.py`. Additionally, I requested its assistance in calculating the Euclidean distance between arrays and filling out dataclasses from a TOML config file that contains a superset of the dataclass parameters.

It suggested potential tests to implement as well, and have been an immense help in formatting the markdown for the README file, as well as the LaTeX for the report, including multiple equations in the line, making pretty code blocks, and formatting the pseudocode.